

周期报文 Offset 与 Com_MainFunctionTx 的 CAN/CPU 平衡联合优化方法

理论推导与算法设计

篠見由紀

2026 年 7 月

目录

1 问题定义与设计目标	2
2 MainFunction 时间栅格的可表示性模型	2
2.1 单条报文的精确可表示条件	2
2.2 一个 MainFunction 分组的最优 TimeBase	3
3 CPU 侧指标：从真实利用率到可计算代理量	3
3.1 真实 CPU 利用率的理想形式	3
3.2 为什么简单的 CPU Proxy 不够合理	4
3.3 固定调用开销 + 组内报文开销模型	4
4 固定 Offset 下的 MainFunction 精确子求解器	5
4.1 条件精确性的目标	5
4.2 相同 D 类型的报文无需拆分	5
4.3 子集组成本	6
4.4 确定性的子集动态规划	6
5 CAN 侧指标：复用现有 Balanced 评价体系	7
6 联合双目标模型	8
7 ϵ-constraint：复用现有 GCLS 的 Pareto 搜索框架	8
7.1 两个锚点	8
7.2 CPU 预算扫描	9

8	Pareto 过滤与默认折中点选择	9
9	完整算法流程	10
10	数值、确定性与实现边界	11
10.1	整数时间与精确比较	11
10.2	确定性	11
10.3	精确性的范围	12
10.4	ρ 的鲁棒性	12
11	最终推荐输出	12
12	结论	12

1 问题定义与设计目标

本节只讨论新增的 Com_MainFunctionTx 联合优化问题, 不改变原有周期报文 Offset 优化的基本边界。目标是: 在保持 CAN 总线释放分布质量的同时, 避免为了少量慢周期报文而让 Com_MainFunctionTx 以过细的时间基准频繁运行, 并最终直接给出可用于工程配置的 MainFunction 分组与 ComMainTxTimeBase 建议。

AUTOSAR Classic Platform R25-11 将 ComMainTxTimeBase 定义为对应 Com_MainFunctionTx 实例连续调用之间的周期; 同时指出 COM 生成器可以基于该 TimeBase 将通信时序参数转换为内部计数器或 tick, 而 COM 内部的具体 timing handling 属于实现相关行为 [1]。因此, 本方法不把下面的整数整除模型声称 AUTOSAR 标准唯一实现, 而把它明确视为工具用于生成**精确可表示、保守且可解释配置建议**的工程模型。

设周期发送报文集合为

$$\mathcal{M} = \{1, 2, \dots, n\}.$$

对每条报文 i 定义:

- $T_i > 0$: 发送周期;
- $O_i \geq 0$: 最终使用的 Offset;
- $w_i > 0$: 现有 CAN 优化器使用的报文权重, 例如 frame_time_us 或 payload_bytes;
- $O_{\max}$: 当前 Offset 搜索上限, 即现有参数 max_offset_ms。

按照当前工程规则:

$$O_i = 0, \quad T_i > O_{\max}. \quad (1)$$

这类慢周期报文不再把 Offset 作为决策变量, 但**仍然必须同时参与 CAN 负载评价和 MainFunction 分组**。对于

$$T_i \leq O_{\max},$$

Offset 仍从现有离散候选集合中选择。

因此, 新增问题不是“先算 Offset, 再随便分 MainFunction”, 而是:

对每一个候选 Offset 向量, 精确计算其最优 MainFunction 分组和 TimeBase, 再以 CAN 质量与 CPU 调度成本两个指标评价该 Offset 向量, 最后寻找二者的折中解。

2 MainFunction 时间栅格的可表示性模型

2.1 单条报文的精确可表示条件

采用统一整数时间单位 u (实现中建议统一转换为整数微秒), 避免对浮点时间直接进行整除或最大公约数运算。

若报文 i 被分配到 MainFunction g , 其 TimeBase 为 $B_g > 0$ 。在本工具采用的**零相位整数 tick 模型**下, 为了使周期与首发 Offset 都能由该 MainFunction 的调用节拍精确表示, 要求

$$B_g \mid T_i, \quad B_g \mid O_i. \quad (2)$$

其中 $a \mid b$ 表示 a 整除 b 。当 $O_i = 0$ 时, 第二个条件天然成立。

于是定义单条报文自身能够支持的最大时间节拍

$$D_i = \gcd(T_i, O_i), \quad (3)$$

并采用约定

$$\gcd(T_i, 0) = T_i. \quad (4)$$

因此, 任意可行 MainFunction TimeBase 必须满足

$$B_g \mid D_i. \quad (5)$$

例如:

$$T_i = 100 \text{ ms}, \quad O_i = 15 \text{ ms} \Rightarrow D_i = 5 \text{ ms},$$

而

$$T_i = 100 \text{ ms}, \quad O_i = 20 \text{ ms} \Rightarrow D_i = 20 \text{ ms}.$$

这说明 Offset 不仅改变 CAN 总线上的释放位置, 也会直接改变该报文可进入的 MainFunction 时间栅格。

2.2 一个 MainFunction 分组的最优 TimeBase

设 MainFunction 分组 $G \subseteq \mathcal{M}$ 。所有组内报文都必须满足式 (2), 因此组内可行 TimeBase 满足

$$B_G \mid D_i, \quad \forall i \in G. \quad (6)$$

所以最大的可行 TimeBase 为

$$B_G^* = \gcd_{i \in G} D_i = \gcd_{i \in G} (T_i, O_i). \quad (7)$$

命题 1: 固定分组后应始终选择式 (7) 给出的最大 TimeBase。如果 CPU 成本随调用频率 $1/B_G$ 单调增加, 则任何 $B_G < B_G^*$ 都只会使 MainFunction 调用更频繁, 而不会增加时序可表示能力, 因此一定被 B_G^* 支配。

这意味着 ComMainTxTimeBase 不需要作为独立搜索变量。只要报文分组确定, TimeBase 就由式 (7) 唯一确定。

3 CPU 侧指标: 从真实利用率到可计算代理量

3.1 真实 CPU 利用率的理想形式

周期任务常用的处理器利用率模型是

$$U = \sum_j \frac{C_j}{T_j}, \quad (8)$$

其中 C_j 是一次执行时间, T_j 是调用周期 [2]。映射到 MainFunction, 可写为

$$U_{\text{COM-Tx}} = \sum_g \frac{C_g}{B_g}, \quad (9)$$

其中 C_g 是 MainFunction g 一次调用的实际 CPU 执行时间。

问题在于: 仅凭 DBC 和工具生成的分组, 通常得不到可靠的 C_g 。因此第一版不能把式 (9) 伪装成“真实 CPU 占用率”, 而需要建立一个结构化、可解释的代理模型。

3.2 为什么简单的 CPU Proxy 不够合理

令 $N_g = |G_g|$ 表示 MainFunction g 中的周期 Tx I-PDU 数量。最直接的代理量是

$$P_0 = \sum_g \frac{N_g}{B_g}. \quad (10)$$

它可以理解为单位时间内的“报文检查机会”数量, 但存在严重的**分裂退化**。

设一个组 G 的 TimeBase 为 B , 把它拆成两个非空子组 G_1, G_2 。由于子集的最大公约数不会比全集更小,

$$B_1 \geq B, \quad B_2 \geq B. \quad (11)$$

于是

$$\frac{N_1}{B_1} + \frac{N_2}{B_2} \leq \frac{N_1 + N_2}{B} = \frac{N}{B}. \quad (12)$$

因此, 仅使用 P_0 时, **拆分 MainFunction 从不使目标变差**, 不断拆分到单报文 MainFunction 往往会得到更低成本。这个指标没有体现“多一个 MainFunction 本身也有固定调用开销”, 因而不能单独作为最终 CPU 指标。

3.3 固定调用开销 + 组内报文开销模型

更合理的近似是把一次 MainFunction 执行时间写成

$$C_g \approx C_0 + C_1 N_g, \quad (13)$$

其中:

- $C_0 > 0$: 进入 MainFunction、固定逻辑、循环框架等与组内报文数无关的固定开销;
- $C_1 > 0$: 每增加一条周期 Tx I-PDU 带来的平均增量处理成本。

代入式 (9):

$$U_{\text{COM-Tx}} \approx \sum_g \frac{C_0 + C_1 N_g}{B_g} = C_1 \sum_g \frac{\rho + N_g}{B_g}, \quad (14)$$

其中

$$\rho = \frac{C_0}{C_1}. \quad (15)$$

由于 $C_1 > 0$ 是所有方案共享的正比例因子，最小化近似 CPU 利用率等价于最小化

$$P_{\text{CPU}}(\Pi) = \sum_{g \in \Pi} \frac{\rho + N_g}{B_g} \quad (16)$$

其中 Π 是 MainFunction 分区。

ρ 的解释非常直接：一次 MainFunction 固定调用开销，相当于多少条 I-PDU 的增量检查开销。

在没有实测数据时，本设计采用

$$\rho = 1 \quad (17)$$

作为默认的单位归一化假设，即“一次固定调用开销与一条 I-PDU 的平均增量检查成本同量级”。它不是物理测量值，因此工具必须把该量称为 **CPU Cost Proxy / CPU 调度成本代理指标**，不能称为实际 CPU 百分比。

若后续获得目标 ECU 测量数据，可拟合

$$C(N) = \beta_0 + \beta_1 N, \quad (18)$$

并直接更新为

$$\rho = \frac{\beta_0}{\beta_1}. \quad (19)$$

这样无需改变优化框架即可从结构代理模型平滑升级到经测量标定的模型。

4 固定 Offset 下的 MainFunction 精确子求解器

4.1 条件精确性的目标

给定一个完整 Offset 向量

$$\mathbf{O} = (O_1, \dots, O_n),$$

所有 D_i 由式 (3) 确定。此时 MainFunction 子问题为：

$$\min_{\Pi} P_{\text{CPU}}(\Pi) = \sum_{G \in \Pi} \frac{\rho + |G|}{\gcd_{i \in G} D_i}. \quad (20)$$

该子问题不再涉及 CAN 指标，因此应使用**确定性精确求解**，而不是随机启发式。这样可以保证：对于同一个 Offset 向量，工具给出的 MainFunction 分组和 CPU Proxy 始终是该模型下的全局最优值。

4.2 相同 D 类型的报文无需拆分

设若干报文拥有相同的 $D_i = d$ 。假设同一 d 类型被拆到两个组，其 TimeBase 分别为 $b_1 \leq b_2$ 。因为两者都必须整除 d ，把一条 d 类型报文从 b_1 组移动到 b_2 组时：

1. 向 b_2 组加入 d 不会降低该组 gcd，因为 $b_2 \mid d$ ；

2. 从 b_1 组删除元素后, 该组 gcd 只可能保持或增大;
3. 报文自身的变动成本由 $1/b_1$ 变成 $1/b_2$, 不会增加;
4. 若原组变空, 还会额外消除一次固定开销。

因此存在一个不劣的解, 把同一 d 类型全部放入可用 TimeBase 最大的同一个组。

命题 2: 至少存在一个全局最优解, 使每一种相同的 D_i 类型整体分配, 不被拆分到多个 MainFunction。这允许把 n 条报文压缩为 m 个不同的 D 类型, 通常 $m \ll n$ 。

设不同类型为

$$d_1, d_2, \dots, d_m,$$

对应报文数量为

$$n_1, n_2, \dots, n_m.$$

4.3 子集组成本

对任意非空类型子集 $A \subseteq \{1, \dots, m\}$, 若这些类型放入同一个 MainFunction, 则

$$B(A) = \gcd_{j \in A} d_j, \quad (21)$$

$$N(A) = \sum_{j \in A} n_j, \quad (22)$$

其最小 CPU 代理成本为

$$c(A) = \frac{\rho + N(A)}{B(A)}. \quad (23)$$

于是原问题转化成对 m 个类型进行集合划分:

$$\min_{\Pi_D} \sum_{A \in \Pi_D} c(A). \quad (24)$$

4.4 确定性的子集动态规划

用 bitmask 表示尚未分组的类型集合 S 。定义 $F(S)$ 为集合 S 的最小 CPU Proxy。边界为

$$F(\emptyset) = 0. \quad (25)$$

为了避免重复枚举无序分区, 令 $p(S)$ 为 S 中编号最小的类型。递推为

$$F(S) = \min_{\substack{A \subseteq S \\ p(S) \in A}} [c(A) + F(S \setminus A)] \quad (26)$$

同时记录使最小值成立的 A , 即可回溯得到完整 MainFunction 分组。

该递推枚举每个集合分区一次, 属于精确算法。最坏时间复杂度为

$$O(3^m), \quad (27)$$

空间复杂度为

$$O(2^m). \quad (28)$$

这里的关键是复杂度取决于不同 D_i 的种类数 m ，而不是报文总数 n 。在本项目的离散周期与 Offset 栅格下， D_i 只能取周期和候选 Offset 的公约数，实际类型数通常很小，因此该精确子求解器具有工程可行性。

实现中应预计算所有子集的 $B(A)$ 、 $N(A)$ 和 $c(A)$ ，并对输入直方图

$$\mathcal{H}(\mathbf{O}) = \{(d_j, n_j)\}_{j=1}^m \quad (29)$$

做缓存。同一个 D 直方图即使来自不同 Offset assignment，其 CPU 精确子问题完全相同，可直接复用结果。

5 CAN 侧指标：复用现有 Balanced 评价体系

新增功能不重新定义 CAN 质量指标，而复用现有 Balanced 模式的核心思想：**Peak 作为不可突破的 guardrail，稳态平方负载 Q_{ss} 作为均匀性目标。**

设稳态超周期为

$$H = \text{lcm}(T_1, \dots, T_n),$$

时隙宽度为 Δ ，第 k 个半开时隙为

$$I_k = [k\Delta, (k+1)\Delta), \quad k = 0, 1, \dots, H/\Delta - 1.$$

报文 i 的周期释放时刻为

$$r_{i,q} = O_i + qT_i. \quad (30)$$

则稳态时隙负载为

$$L_k(\mathbf{O}) = \sum_i w_i \sum_q \mathbf{1}(r_{i,q} \in I_k). \quad (31)$$

CAN 均匀性指标保持为

$$Q_{ss}(\mathbf{O}) = \sum_k L_k(\mathbf{O})^2 \quad (32)$$

越小表示负载越均匀地分散到不同时间槽。

峰值可用

$$Z_{ss}(\mathbf{O}) = \max_k L_k(\mathbf{O}) \quad (33)$$

理解；实际代码若已经采用更完整的 Peak 词典序 guardrail（例如现有的违规计数、违规量和峰值组合），**必须原样复用现有定义，不在本功能中另造一套 Peak 判据。**

若现有 Balanced 允许相对峰值容差 τ ，则可以抽象写成

$$\mathbf{O} \in \mathcal{F}_{\text{peak}}, \quad (34)$$

其中 $\mathcal{F}_{\text{peak}}$ 表示满足现有 Peak 预算的 Offset 可行集合。

特别注意：式 (1) 中被固定为 $O_i = 0$ 的慢周期报文仍然进入式 (31)–(33)。它们只是失去 Offset 决策自由度，绝不能从 CAN 评价中删除。

6 联合双目标模型

对每个 Offset 向量 $\mathbf{O}$ ，先调用第 4 节的精确子求解器，定义

$$\Phi(\mathbf{O}) = \min_{\Pi} P_{\text{CPU}}(\Pi | \mathbf{O}). \quad (35)$$

$\Phi(\mathbf{O})$ 是该 Offset assignment 在 MainFunction 最优分组下的最小 CPU Proxy。

于是整个新问题可以压缩为只对 Offset 向量进行外层搜索：

$$\min_{\mathbf{O} \in \mathcal{T}_{\text{peak}}} (Q_{ss}(\mathbf{O}), \Phi(\mathbf{O})) \quad (36)$$

并附加工程规则：

$$O_i = 0 \quad \text{if } T_i > O_{\max}, \quad (37)$$

其余报文的 O_i 来自现有 Offset 候选集合。

这种分层结构有一个重要性质：对于同一个 $\mathbf{O}$ ，所有 MainFunction 分组的 CAN 指标完全相同。因此任何不是 CPU 最优的分组都必然被同一 Offset 下的精确最优分组支配，没有必要把“报文属于哪个 MainFunction”直接塞进 GCLS 外层搜索变量中。

最终搜索结构为：

$$\mathbf{O} \longrightarrow \begin{cases} Q_{ss}(\mathbf{O}), \text{ Peak guardrail,} \\ \Phi(\mathbf{O}) \text{ 及其精确 MainFunction 分组} \end{cases} \quad (38)$$

7 ϵ -constraint：复用现有 GCLS 的 Pareto 搜索框架

直接构造

$$\alpha Q_{ss} + \beta \Phi$$

会引入难以解释的跨量纲权重。更合适的是采用 ϵ -constraint 思路：保留一个目标作为优化目标，把另一个目标转换为预算约束，通过扫描预算得到非支配解集 [3]。

为了最大限度复用现有 Balanced/GCLS 的 Q_{ss} 搜索能力，本设计选择：

以 Q_{ss} 为主目标，以 CPU Proxy 为 ϵ 约束。

7.1 两个锚点

先得到 CAN 锚点

$$\mathbf{O}^Q = \arg \min_{\mathbf{O} \in \mathcal{T}_{\text{peak}}} Q_{ss}(\mathbf{O}), \quad (39)$$

同 Q_{ss} 时以更小的 Φ 作为次级判据。记

$$Q_{\min} = Q_{ss}(\mathbf{O}^Q), \quad P_Q = \Phi(\mathbf{O}^Q). \quad (40)$$

再得到 CPU 锚点

$$\mathbf{O}^P = \arg \min_{\mathbf{O} \in \mathcal{T}_{\text{peak}}} \Phi(\mathbf{O}), \quad (41)$$

同 Φ 时以更小的 Q_{ss} 作为次级判据。记

$$P_{\min} = \Phi(\mathbf{O}^P), \quad Q_P = Q_{ss}(\mathbf{O}^P). \quad (42)$$

若 $P_Q = P_{\min}$ 或 $Q_P = Q_{\min}$, 说明当前实例在两个目标之间不存在可观测冲突, 可直接选择同时最优的方案, 不需要继续扫描。

7.2 CPU 预算扫描

默认采用 $J = 20$ 个区间, 即共 21 个 CPU 预算点:

$$\varepsilon_j = P_{\min} + \frac{j}{J}(P_Q - P_{\min}), \quad j = 0, 1, \dots, J. \quad (43)$$

对每个预算解:

$$\begin{array}{ll} \min_{\mathbf{O}} & Q_{ss}(\mathbf{O}) \\ \text{s.t.} & \mathbf{O} \in \mathcal{F}_{\text{peak}}, \\ & \Phi(\mathbf{O}) \leq \varepsilon_j, \\ & O_i = 0, \quad T_i > O_{\max}. \end{array} \quad (44)$$

在 Q_{ss} 相同的情况下, 再最小化 Φ , 形成词典序目标

$$(Q_{ss}, \Phi). \quad (45)$$

这可以避免保留明显弱化的解, 并与 augmented ε -constraint 的思想一致。

扫描顺序从严格 CPU 端

$$\varepsilon_0 = P_{\min}$$

逐步放宽到

$$\varepsilon_J = P_Q.$$

这样上一预算点得到的解在下一预算中仍然可行, 可作为 GCLS warm start; 理论上随着 CPU 预算放宽, 最佳可达 Q_{ss} 不应变差。

由于外层 Offset 求解仍由 GCLS 等启发式完成, 式 (44) 得到的是**搜索到的 Pareto 前沿近似**, 而不是全局精确 Pareto 前沿。精确性只对“给定 Offset 后的 MainFunction 分组子问题”成立。文档和 GUI 均不得把整体结果表述为数学全局最优。

8 Pareto 过滤与默认折中点选择

收集两个锚点和全部 ε 扫描结果, 先按 (Q_{ss}, Φ) 去重, 再删除被其他方案支配的点。若方案 a, b 满足

$$Q_a \leq Q_b, \quad P_a \leq P_b, \quad (46)$$

且至少一个严格小于, 则 a 支配 b , b 不进入最终集合。

设过滤后的非支配点为

$$\mathcal{P} = \{(Q_k, P_k)\}_{k=1}^K.$$

为了消除两个指标的量纲差异，采用极值归一化：

$$\hat{Q}_k = \frac{Q_k - Q_{\min}}{Q_{\max} - Q_{\min}}, \quad (47)$$

$$\hat{P}_k = \frac{P_k - P_{\min}}{P_{\max} - P_{\min}}. \quad (48)$$

其中 $Q_{\max}$ 和 $P_{\max}$ 取最终非支配集合中的最大值。归一化后两个目标均满足“0 最好、1 最差”。

对于典型的两个极端点 (0, 1) 与 (1, 0)，连接二者的弦为

$$\hat{Q} + \hat{P} = 1. \quad (49)$$

定义每个点朝理想点 (0, 0) 方向偏离该弦的几何膝点得分

$$\kappa_k = \frac{1 - \hat{Q}_k - \hat{P}_k}{\sqrt{2}}. \quad (50)$$

优先选择

$$k^* = \arg \max_k \kappa_k. \quad (51)$$

其含义是：该点相对两个极端方案，获得了最大的“同时改善空间”，通常对应继续改善其中一个指标时，另一个指标的边际代价开始明显增大的位置。膝点作为多目标系统中的折中选择具有成熟的工程背景 [4]。

若所有 $\kappa_k \leq 0$ ，或者多个点的 κ 在数值容差内相同，则不强行宣称存在明显 knee，而退化为选择离理想点最近的方案：

$$k^* = \arg \min_k \sqrt{\hat{Q}_k^2 + \hat{P}_k^2}. \quad (52)$$

最终确定性 tie-break 顺序为：

1. 更小的理想点距离；
2. 更少的 MainFunction 数量；
3. 更小的 Q_{ss} ；
4. 更小的 CPU Proxy；
5. 稳定的 Offset assignment / group signature 字典序。

9 完整算法流程

最终采用以下单一路径，不再增加“CPU 优先”等额外模式：

1. 读取所有最终参与分析的周期 Tx 报文。
2. 对 $T_i > O_{\max}$ 的报文固定 $O_i = 0$ ；其余报文保留现有离散 Offset 候选集合。

3. 沿用现有 Peak/Balanced 评价器计算每个 Offset assignment 的 Peak 和 Q_{ss} 。
4. 对每个 Offset assignment 计算

$$D_i = \gcd(T_i, O_i).$$

5. 按 D_i 聚合计数, 形成类型直方图。
6. 通过式 (26) 的确定性子集 DP 精确求解 MainFunction 最优分组, 得到每组

$$B_g = \gcd_{i \in g}(T_i, O_i)$$

以及精确 CPU Proxy

$$\Phi(\mathbf{O}) = \sum_g \frac{\rho + N_g}{B_g}.$$

7. 先求 CAN 锚点与 CPU 锚点。
8. 按式 (43)–(44) 执行 21 点 CPU ϵ 预算扫描; 每个预算继续使用 GCLS 搜索 Offset, 不直接把 MainFunction 归属作为外层变量。
9. 汇总全部候选, 去重并做 Pareto 支配过滤。
10. 按式 (47)–(52) 自动选择默认 knee 方案。
11. 输出推荐的 Offset、MainFunction 分组、每组 ComMainTxTimeBase、 Q_{ss} 、CPU Proxy、Pareto 点集和推荐点选择依据。

10 数值、确定性与实现边界

10.1 整数时间与精确比较

所有 T_i, O_i, B_g, D_i 必须先统一为整数微秒或统一内部 tick。禁止用二进制浮点数直接做 gcd、整除可行性判断。

CPU Proxy 是有理数。精确子求解器内部应使用有理数或等价的整数缩放比较; 只有 GUI 展示、曲线绘制和膝点归一化阶段才允许转换为浮点数。

10.2 确定性

给定同一 Offset assignment、同一 ρ 和同一输入集合, 精确子求解器必须得到相同的:

- CPU Proxy;
- MainFunction 数量;
- 每组 TimeBase;
- 类型到 MainFunction 的归属。

若存在多个 CPU Proxy 完全相同的分区, 依次选择 MainFunction 数更少、TimeBase 序列按降序字典序更大、类型签名字典序更稳定的方案。

10.3 精确性的范围

需要明确区分：

- **条件精确：**给定 Offset 向量后，MainFunction 分区由子集 DP 全局精确求解；
- **启发式：**Offset 外层仍依赖 GCLS，因此 CAN/CPU Pareto 集合是当前搜索预算下的近似前沿；
- **代理模型：**未标定 C_0, C_1 时，CPU 指标是结构成本 proxy，而不是真实 CPU 百分比。

10.4 ρ 的鲁棒性

默认 $\rho = 1$ 。为了避免默认结构假设偶然决定结果，实验验证至少应补充

$$\rho \in \{0.5, 1, 2, 4\} \quad (53)$$

的敏感性扫描。若 knee 方案的 MainFunction 数量、主要 TimeBase 和 Offset 结构在该范围内基本稳定，可以认为推荐对固定调用成本假设具有较好鲁棒性；若结果剧烈变化，应明确标记“CPU 模型敏感”，并优先通过 ECU 实测标定式 (18)。

11 最终推荐输出

单个网段最终至少输出：

- 推荐 Offset assignment；
- 推荐 MainFunction 数量；
- 每个 MainFunction 的 ComMainTxTimeBase；
- 每条周期 Tx I-PDU 的建议 MainFunction 归属；
- CAN 指标：现有 Peak guardrail 状态、 Q_{ss} ；
- CPU 指标： P_{CPU} 及使用的 ρ ；
- ϵ 扫描得到的非支配点列表；
- 自动选中的 knee 点及归一化 $(\hat{Q}, \hat{P})$ ；
- 对 $T_i > O_{\max}$ 的报文明示“Offset 固定为 0，但仍参与 CAN 与 CPU 评价”。

推荐配置应明确表述为工具生成的 **MainFunction/TimeBase 建议**，而不是对现有 ECU ARXML 配置的反推结果。

12 结论

本方法把原本耦合度很高的“Offset + MainFunction 归属 + TimeBase”三类变量重新分层：外层只搜索 Offset，内层对每个 Offset assignment 精确求解 MainFunction 分组。CAN 侧复用现有 Peak

+ Q_{ss} Balanced 指标, CPU 侧从真实利用率 $\sum C_g/B_g$ 出发, 推导出包含固定调用成本的结构代理量 $\sum(\rho + N_g)/B_g$, 避免了 $\sum N_g/B_g$ 对过度拆分 MainFunction 的系统性偏好。最终通过 ϵ -constraint 扫描获得 CAN/CPU 非支配解集, 并用归一化几何 knee 规则自动给出默认折中方案。

该结构的核心优点是: MainFunction 子问题在给定 Offset 后可以做到确定性、可解释、全局精确; 同时又最大程度保留现有 GCLS/Balanced 搜索框架, 不需要把 MainFunction 分组本身扩张为外层组合搜索变量。

参考文献

- [1] AUTOSAR, *Specification of Communication*, Classic Platform R25-11, Document ID 15, 2025.
https://www.autosar.org/fileadmin/standards/R25-11/CP/AUTOSAR_CP_SWS_COM.pdf
- [2] C. L. Liu and J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973. DOI: <https://doi.org/10.1145/321738.321743>.
- [3] G. Mavrotas, “Effective implementation of the ϵ -constraint method in Multi-Objective Mathematical Programming problems,” *Applied Mathematics and Computation*, vol. 213, no. 2, pp. 455–465, 2009. DOI: <https://doi.org/10.1016/j.amc.2009.03.037>.
- [4] V. Satopaa, J. Albrecht, D. Irwin, and B. Raghavan, “Finding a Kneedle in a Haystack: Detecting Knee Points in System Behavior,” *31st International Conference on Distributed Computing Systems Workshops*, pp. 166–171, 2011. DOI: <https://doi.org/10.1109/ICDCSW.2011.20>.